FastAPI Ad Copy Microservice

Comprehensive End-to-End Technical Documentation & Code Reference

Author:	Kamran Rehman	**Architecture:**	FastAPI + Async Anthropic SDK
Model:	Claude 3.5 Sonnet	**Repository:**	ad-copy-generator-microservice

1. Executive Summary & Purpose

This microservice was built to address a common high-volume e-commerce challenge: automatically generating highly targeted, multi-angled marketing copy for product catalogs. Operating as a lightweight asynchronous HTTP microservice powered by Python and FastAPI, it routes structured product input attributes directly to Anthropic's Claude 3.5 Sonnet engine and returns strict, validated JSON ad variants across four distinct copywriting frameworks (Pain Point, Social Proof, Feature Focus, and Urgency).

2. Environment Setup & Dependency Rationale

Each library and environment choice was selected deliberately to ensure production performance, security, and developer ergonomics:

Component / Package	Installation Command	Technical Purpose & Justification
Python Virtual Env (venv)	`python -m venv venv`	Creates an isolated Python runtime, preventing package pollution across global system interpreters and ensuring repeatable deployments.
FastAPI	`pip install fastapi`	High-performance web framework built on Starlette and Pydantic. Enables asynchronous request handlers, automatic Swagger UI documentation, and native schema validation.
Uvicorn	`pip install uvicorn`	An ASGI (Asynchronous Server Gateway Interface) web server implementation used to run FastAPI applications in production and local environments.
Anthropic SDK	`pip install anthropic`	Official Python client library to interface asynchronously with the Claude 3.5 Sonnet API using typed request models.
Pydantic	`pip install pydantic`	Data validation library enforcing strict runtime data typing for API request payloads and LLM responses.

Component / Package	Installation Command	Technical Purpose & Justification
Python-dotenv	`pip install python-dotenv`	Loads environment variables from a local `.env` file into system memory, preventing API keys from being checked into source control.

3. Secure Configuration & Git Workflow

To enforce strict security standards, API credentials are isolated from source code. The project utilizes a `.env.example` template tracked by Git, while the actual secret file `.env` is ignored via `.gitignore`.

> **Security Verification Protocol:** During deployment, `.gitignore` explicit rules (`.env`, `venv/`, `__pycache__/`) ensure that zero private API credentials (e.g., `sk-ant-api03-...`) leak to public GitHub repositories.

4. Comprehensive Code Breakdown

Below is the line-by-line technical breakdown of the key backend files comprising the microservice architecture.

Module A: Data Models (`app/models.py`)

Source Code Line	Line-by-Line Explanation
`from pydantic import BaseModel, Field`	Imports Pydantic's core class for defining data schema contracts and Field for metadata validation.
`class AdRequest`	Defines the inbound JSON request schema expected from external callers.
`  product_name: str = Field(..., example="Wireless Earbuds")`	Enforces required string input for the product title with Swagger UI documentation example.
`"ANC" "24h battery"`	Requires an array of strings detailing key product capabilities.
`  target_audience: str = Field(..., example="Fitness Enthusiasts")`	Captures target demographic to allow Claude to adapt tone and messaging.
`class AdResponse`	Defines the structured schema returned back to the API client.
`  pain_point_angle: str`	Ad copy variant targeting customer pain points and direct solutions.
	Ad copy variant emphasizing testimonials, popular adoption, or review vibes.
`  feature_focus_angle: str`	Ad copy variant focusing on specifications and technical capabilities.
	Ad copy variant utilizing call-to-action (CTA) and scarcity drivers.

Module B: Generation Service (`app/generator.py`)

Source Code Line	Line-by-Line Explanation

`import json, os`
 	
`from anthropic import AsyncAnthropic`
	Imports standard JSON/OS libraries and Anthropic's non-blocking asynchronous SDK client.

`"ANTHROPIC_API_KEY"`	Instantiates global async client pulling credentials dynamically from loaded environment variables.
`async def generate_ad_copy(req: AdRequest) -> AdResponse:`	Asynchronous generation function accepting typed request parameters and returning typed output.
`"Generate 4 ad angles for {req.product_name}..."`	Constructs system prompt ordering Claude to return raw JSON matching the exact schema keys.
`res = await client.messages.create(...)`	Non-blocking await call executing API request against `claude-3-5-sonnet-20241022`.
	Parses the raw JSON string response returned inside the message content payload block.
`return AdResponse(**data)`	Unpacks parsed dictionary into Pydantic model to validate attributes prior to returning.

5. Troubleshooting History & Error Resolution Log

During local setup, API integration, and GitHub deployment, several environment and command execution errors were encountered. Below is the systematic audit of each issue and its fix:

Error Symptom / Log	Root Cause Analysis	Implemented Resolution
`Application Control policy blocked pip.exe`	Windows AppLocker / WDAC security policy prevented direct execution of binary `pip.exe` inside PowerShell.	Bypassed policy by invoking pip directly through the active Python interpreter: `python -m pip freeze > requirements.txt`
`git : The term 'git' is not recognized...`	Git for Windows binary was missing from system environment PATH variable or not installed.	Installed Git for Windows via installer, refreshed system environment variables, and launched a fresh PowerShell instance.
`Author identity unknown / unable to auto-detect email`	Git requires user identity configuration to attribute commit authors in repository logs.	Configured local repository author metadata using: `git config user.name "Kamran"` `git config user.email "kamran@..."`

Error Symptom / Log	Root Cause Analysis	Implemented Resolution
`fatal: unable to access 'https://github.com//...' Error 400`	Git remote origin URL contained literal generic template placeholders rather than valid GitHub username.	Updated remote pointer using: `git remote set-url origin https://github.com/kamranrehman997/...`
`remote: Repository not found. fatal: repository ... not found`	Attempted to execute `git push` before creating the corresponding empty repository under the GitHub user account.	Navigated to `github.com/new`, initialized empty repository matching the exact name, and re-ran `git push -u origin main`.

6. Final Deployment Verification

The service was successfully pushed to GitHub with all files intact:

Repository URL: https://github.com/kamranrehman997/ad-copy-generator-microservice

The microservice is now fully production-ready, featuring full API key isolation, asynchronous performance execution, automatic OpenAPI schema generation, and clean portfolio packaging.